МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:

Д. А. Марковский

студент группы ПО-13

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ

27.03.2026

Брест 2026

Цель работы:

Приобрести практические навыки разработки API и баз данных.

Задание:

5) База данных: Сборка компьютера.
Реализовать базу данных из не менее 5 таблиц на заданную тематику.
При реализации продумать типизацию полей и внешние ключи в таблицах;
Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними;
На языке Python с использованием SQLAlchemy реализовать подключение к БД.
Реализовать основные операции с данными (выборку, добавление, удаление, модификацию). Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт.

Код программы:

sqalchemy_db.py:

```
# pylint: disable=too-few-public-methods
from datetime import date
from sqlalchemy import create_engine, Column, Integer, String, Float, Date,
ForeignKey
from sqlalchemy.orm import declarative_base, relationship, sessionmaker

engine = create_engine('sqlite:///computer_builds.db', echo=True)
Base = declarative_base()


class Manufacturer(Base):
    __tablename__ = 'manufacturers'

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    country = Column(String(50))
    founded_year = Column(Integer)

    components = relationship('Component', back_populates='manufacturer')


class Category(Base):
    __tablename__ = 'categories'

    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    description = Column(String)

    components = relationship('Component', back_populates='category')


class Component(Base):
    __tablename__ = 'components'

    id = Column(Integer, primary_key=True)
    name = Column(String(200), nullable=False)
    price = Column(Float, nullable=False)
    manufacturer_id = Column(Integer, ForeignKey('manufacturers.id',
ondelete='SET NULL'))
    category_id = Column(Integer, ForeignKey('categories.id', ondelete='SET
```

```
NULL'))
    release_date = Column(Date)
    stock_quantity = Column(Integer, default=0)

    manufacturer = relationship('Manufacturer', back_populates='components')
    category = relationship('Category', back_populates='components')
    builds = relationship('BuildComponent', back_populates='component')


class Build(Base):
    __tablename__ = 'builds'

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    build_date = Column(Date, default=date.today)
    total_price = Column(Float)
    purpose = Column(String(50))

    components = relationship('BuildComponent', back_populates='build')


class BuildComponent(Base):
    __tablename__ = 'build_components'

    id = Column(Integer, primary_key=True)
    build_id = Column(Integer, ForeignKey('builds.id', ondelete='CASCADE'),
nullable=False)
    component_id = Column(Integer, ForeignKey('components.id',
ondelete='CASCADE'), nullable=False)
    quantity = Column(Integer, default=1)

    build = relationship('Build', back_populates='components')
    component = relationship('Component', back_populates='builds')


Base.metadata.create_all(engine)
SESSION_MAKER = sessionmaker(bind=engine)
session = SESSION_MAKER()

print("Подключение к БД через SQLAlchemy успешно установлено!")


def add_manufacturer(name, country=None, founded_year=None):
    manufacturer = Manufacturer(name=name, country=country,
founded_year=founded_year)
    session.add(manufacturer)
    session.commit()
    print(f"Производитель {name} добавлен с id {manufacturer.id}")
    return manufacturer


def add_category(name, description=None):
    category = Category(name=name, description=description)
    session.add(category)
    session.commit()
    print(f"Категория {name} добавлена с id {category.id}")
    return category


def add_component(name, price, manufacturer_id=None, category_id=None,
stock_quantity=0):
    component = Component(
```

```
        name=name,
        price=price,
        manufacturer_id=manufacturer_id,
        category_id=category_id,
        stock_quantity=stock_quantity
    )
    session.add(component)
    session.commit()
    print(f"Компонент {name} добавлен с id {component.id}")
    return component


def add_build(name, purpose=None, total_price=None):
    build = Build(name=name, purpose=purpose, total_price=total_price)
    session.add(build)
    session.commit()
    print(f"Сборка {name} добавлена с id {build.id}")
    return build


def add_component_to_build(build_id, component_id, quantity=1):
    build_component = BuildComponent(
        build_id=build_id,
        component_id=component_id,
        quantity=quantity
    )
    session.add(build_component)
    session.commit()
    print(f"Компонент {component_id} добавлен в сборку {build_id} (кол-во:
{quantity})")
    return build_component


def get_all_manufacturers():
    manufacturers = session.query(Manufacturer).all()
    print("ВСЕ ПРОИЗВОДИТЕЛИ")
    for m in manufacturers:
        print(f"ID: {m.id}, Название: {m.name}, Страна: {m.country}")
    return manufacturers


def get_all_components():
    components = session.query(Component).all()
    print("ВСЕ КОМПОНЕНТЫ")
    for c in components:
        print(f"ID: {c.id}, Название: {c.name}, Цена: {c.price}, Кол-во:
{c.stock_quantity}")
    return components


def get_components_by_category(category_id):
    components = session.query(Component).filter(Component.category_id ==
category_id).all()
    print(f"КОМПОНЕНТЫ КАТЕГОРИИ {category_id}")
    for c in components:
        print(f"ID: {c.id}, Название: {c.name}, Цена: {c.price}")
    return components


def get_build_details(build_id):
    build = session.query(Build).filter(Build.id == build_id).first()
    if build:
```

```
        print(f"СБОРКА: {build.name}")
        print(f"ID: {build.id}, Дата: {build.build_date}, Цель: {build.purpose}")
        print("Компоненты:")
        total = 0
        for bc in build.components:
            comp = bc.component
            subtotal = comp.price * bc.quantity
            total += subtotal
            print(f"- {comp.name} x{bc.quantity} = {comp.price} * {bc.quantity} =
{subtotal} руб.")
        print(f"ИТОГО: {total} руб.")
        return build
    return None


def update_component_price(component_id, new_price):
    component = session.query(Component).filter(Component.id ==
component_id).first()
    if component:
        old_price = component.price
        component.price = new_price
        session.commit()
        print(f"Цена компонента {component.name} изменена с {old_price} на
{new_price}")
    else:
        print(f"Компонент с id {component_id} не найден")


def update_manufacturer_country(manufacturer_id, new_country):
    manufacturer = session.query(Manufacturer).filter(Manufacturer.id ==
manufacturer_id).first()
    if manufacturer:
        old_country = manufacturer.country
        manufacturer.country = new_country
        session.commit()
        print(f"Страна производителя {manufacturer.name} изменена с {old_country}
на {new_country}")
    else:
        print(f"Производитель с id {manufacturer_id} не найден")


def update_component_quantity(component_id, new_quantity):
    component = session.query(Component).filter(Component.id ==
component_id).first()
    if component:
        component.stock_quantity = new_quantity
        session.commit()
        print(f"Количество компонента {component.name} на складе обновлено до
{new_quantity}")
    else:
        print(f"Компонент с id {component_id} не найден")


def delete_manufacturer(manufacturer_id):
    manufacturer = session.query(Manufacturer).filter(Manufacturer.id ==
manufacturer_id).first()
    if manufacturer:
        name = manufacturer.name
        session.delete(manufacturer)
        session.commit()
        print(f"Производитель {name} удален")
    else:
```

```
        print(f"Производитель с id {manufacturer_id} не найден")


def delete_component(component_id):
    component = session.query(Component).filter(Component.id ==
component_id).first()
    if component:
        name = component.name
        session.delete(component)
        session.commit()
        print(f"Компонент {name} удален")
    else:
        print(f"Компонент с id {component_id} не найден")


def delete_build(build_id):
    build = session.query(Build).filter(Build.id == build_id).first()
    if build:
        name = build.name
        session.delete(build)
        session.commit()
        print(f"Сборка {name} удалена")
    else:
        print(f"Сборка с id {build_id} не найдена")
```

main.py

```
from datetime import date
from typing import Optional, List
import uvicorn
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from sqlalchemy_db import (
    add_manufacturer, get_all_manufacturers, update_manufacturer_country,
delete_manufacturer,
    add_category,
    add_component, get_all_components, get_components_by_category,
    update_component_price, update_component_quantity, delete_component,
    add_build, get_build_details, delete_build,
    add_component_to_build
)

app = FastAPI(title="Компьютерная сборка API")


class ManufacturerCreate(BaseModel):
    name: str
    country: Optional[str] = None
    founded_year: Optional[int] = None


class ManufacturerResponse(ManufacturerCreate):
    id: int


class CategoryCreate(BaseModel):
    name: str
    description: Optional[str] = None


class CategoryResponse(CategoryCreate):
    id: int


class ComponentCreate(BaseModel):
    name: str
```

```
    price: float
    manufacturer_id: Optional[int] = None
    category_id: Optional[int] = None
    stock_quantity: int = 0


class ComponentResponse(ComponentCreate):
    id: int


class BuildCreate(BaseModel):
    name: str
    purpose: Optional[str] = None
    total_price: Optional[float] = None


class BuildResponse(BuildCreate):
    id: int
    build_date: date


class BuildComponentCreate(BaseModel):
    build_id: int
    component_id: int
    quantity: int = 1


class BuildComponentResponse(BuildComponentCreate):
    id: int


class BuildDetailResponse(BuildResponse):
    components: List[dict]


@app.post("/manufacturers/", response_model=ManufacturerResponse)
def create_manufacturer(manufacturer: ManufacturerCreate):
    try:
        new_manufacturer = add_manufacturer(
            name=manufacturer.name,
            country=manufacturer.country,
            founded_year=manufacturer.founded_year
        )
        return new_manufacturer
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.get("/manufacturers/", response_model=List[ManufacturerResponse])
def read_manufacturers():
    manufacturers = get_all_manufacturers()
    return manufacturers


@app.put("/manufacturers/{manufacturer_id}/country")
def update_manufacturer_country_endpoint(manufacturer_id: int, new_country: str):
    try:
        update_manufacturer_country(manufacturer_id, new_country)
        return {"message": "Страна производителя обновлена"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.delete("/manufacturers/{manufacturer_id}")
def delete_manufacturer_endpoint(manufacturer_id: int):
    try:
        delete_manufacturer(manufacturer_id)
```

```
        return {"message": "Производитель удален"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.post("/categories/", response_model=CategoryResponse)
def create_category(category: CategoryCreate):
    try:
        new_category = add_category(
            name=category.name,
            description=category.description
        )
        return new_category
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.post("/components/", response_model=ComponentResponse)
def create_component(component: ComponentCreate):
    try:
        new_component = add_component(
            name=component.name,
            price=component.price,
            manufacturer_id=component.manufacturer_id,
            category_id=component.category_id,
            stock_quantity=component.stock_quantity
        )
        return new_component
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.get("/components/", response_model=List[ComponentResponse])
def read_components():
    components = get_all_components()
    return components


@app.get("/components/category/{category_id}",
response_model=List[ComponentResponse])
def read_components_by_category(category_id: int):
    components = get_components_by_category(category_id)
    return components


@app.put("/components/{component_id}/price")
def update_component_price_endpoint(component_id: int, new_price: float):
    try:
        update_component_price(component_id, new_price)
        return {"message": "Цена компонента обновлена"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.put("/components/{component_id}/quantity")
def update_component_quantity_endpoint(component_id: int, new_quantity: int):
    try:
        update_component_quantity(component_id, new_quantity)
        return {"message": "Количество компонента обновлено"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.delete("/components/{component_id}")
def delete_component_endpoint(component_id: int):
    try:
        delete_component(component_id)
```

```
        return {"message": "Компонент удален"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.post("/builds/", response_model=BuildResponse)
def create_build(build: BuildCreate):
    try:
        new_build = add_build(
            name=build.name,
            purpose=build.purpose,
            total_price=build.total_price
        )
        return new_build
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.get("/builds/{build_id}", response_model=BuildDetailResponse)
def read_build_details(build_id: int):
    build = get_build_details(build_id)
    if build:
        build_dict = {
            "id": build.id,
            "name": build.name,
            "build_date": build.build_date,
            "total_price": build.total_price,
            "purpose": build.purpose,
            "components": []
        }
        for bc in build.components:
            build_dict["components"].append({
                "component_id": bc.component.id,
                "component_name": bc.component.name,
                "quantity": bc.quantity,
                "price": bc.component.price
            })
        return build_dict
    raise HTTPException(status_code=404, detail="Сборка не найдена")


@app.delete("/builds/{build_id}")
def delete_build_endpoint(build_id: int):
    try:
        delete_build(build_id)
        return {"message": "Сборка удалена"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


@app.post("/build-components/", response_model=BuildComponentResponse)
def add_component_to_build_endpoint(bc: BuildComponentCreate):
    try:
        new_bc = add_component_to_build(
            build_id=bc.build_id,
            component_id=bc.component_id,
            quantity=bc.quantity
        )
        return new_bc
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e)) from e


if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=8000)
    #   http://127.0.0.1:8000/docs
    #   http://127.0.0.1:8000/redoc
```

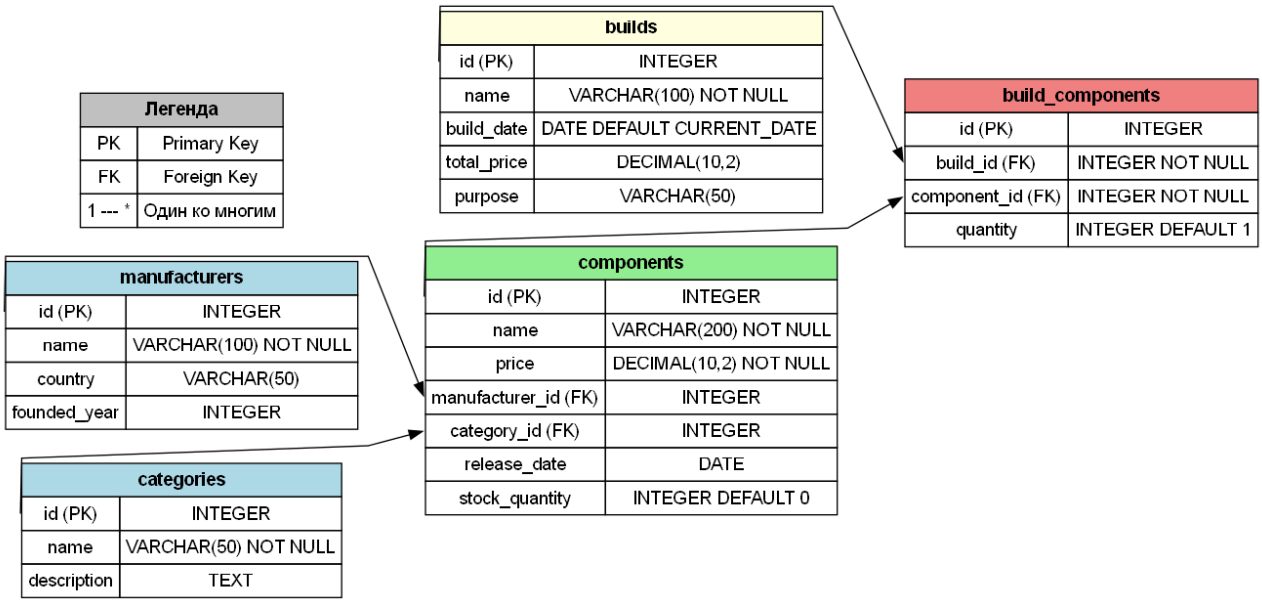
Спецификация ввода

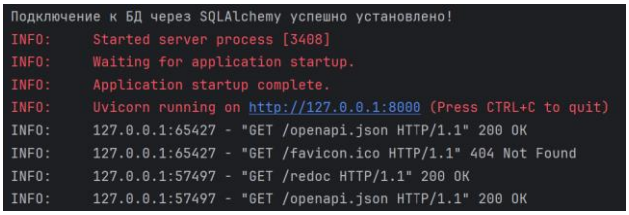
Ввод не требуется. После запуска программы в браузер следует вставить одну из ссылок `http://127.0.0.1:8000/docs` или `http://127.0.0.1:8000/redoc`.

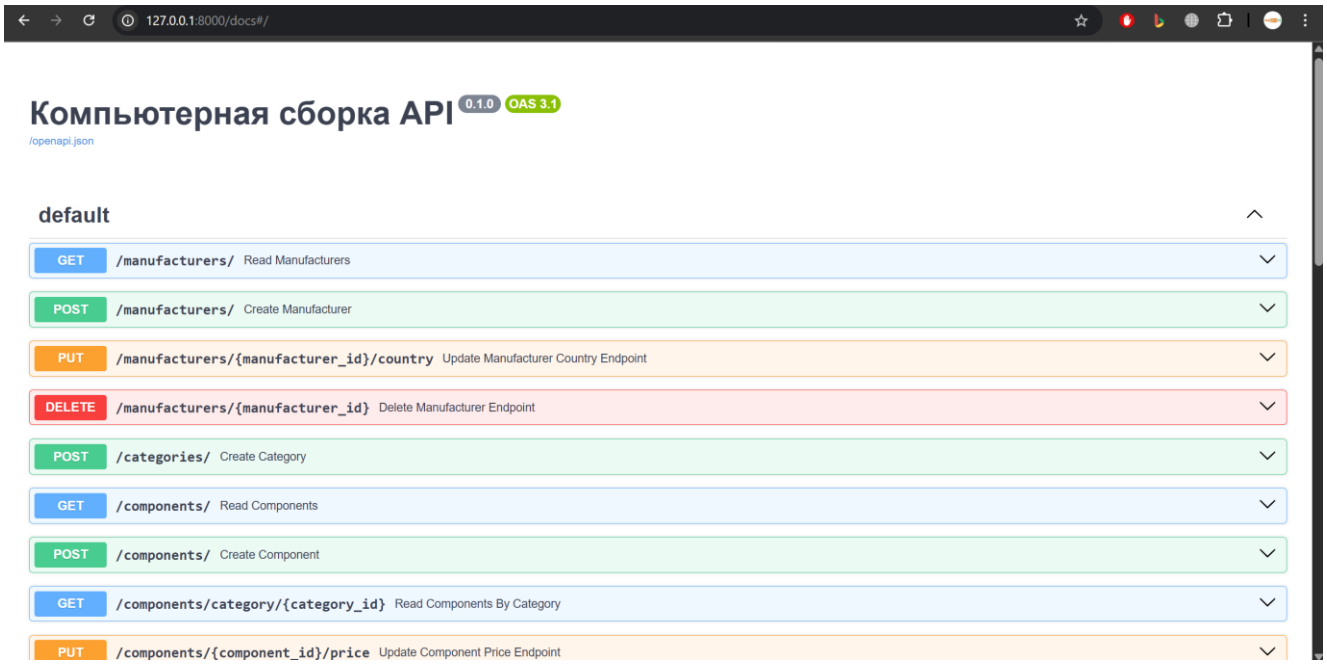
Спецификация вывода

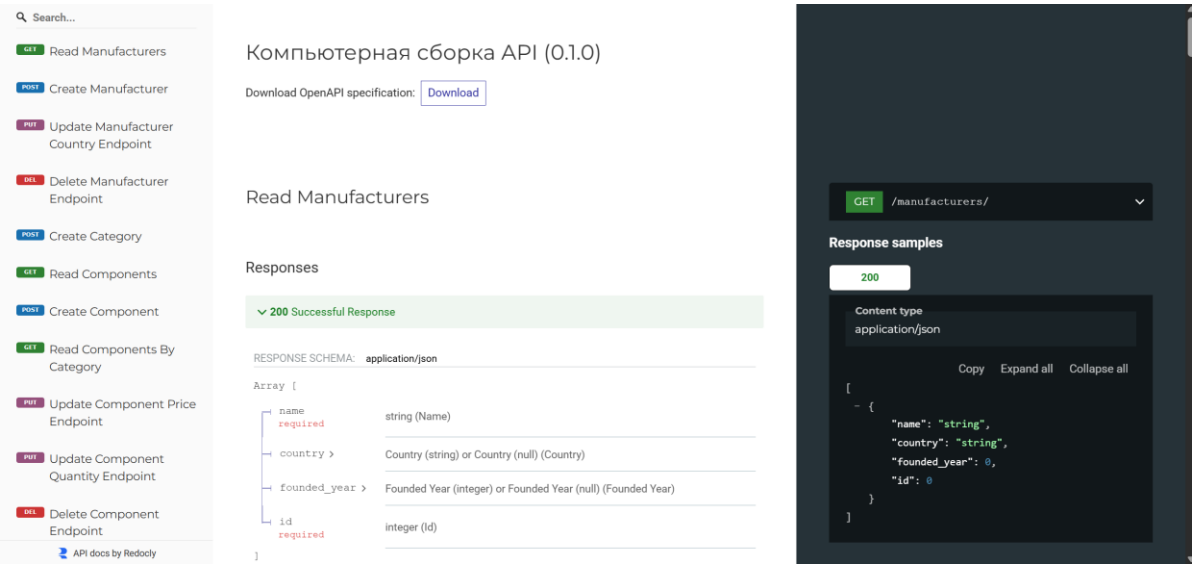
После перехода по ссылке мы перейдём на автоматическую документацию со всеми эндпоинтами, которые можно тестировать.

Рисунки с результатами работы программы









Вывод:

Я приобрел практические навыки разработки API и баз данных.